CS3230 : Design and Analysis of Algorithms (Spring 2014)

Tutorial Set #4

[For discussion during Week 6]

OUT: 14-Feb-2014 **Tutorials:** Mon & Thu, 17, 20 Feb 2014

Topics: Sorting, Lower Bound, Linear Time Sorting, Selection, Max, Min, Max-Min, Rank, Augmenting Data Structures, what else?

IMPORTANT: Read "Remarks about Homework" – also applies to tutorials.

Prepare your answers to all the D-Problems in every tutorial set.

When preparing to present your answers,

- Think of a CLEAR EXPLANATION
- Illustrate with a good worked example;
- Describe the main ideas,
- Can you sketch why the solution works;
- Give analysis of running time, if appropriate
- Can you think of other (perhaps simpler) solutions?

Helpful Hints Series: In the haste to send out this tutorial, Prof. Leong is taking a break on this hint series. Will restart this series after the semester break.

Routine Practice Problems *-- do not turn these in -- but make sure you know how to do them.*

R1. (Exercise 8.2-1 on p. 196 of [CLRS]) [Exercising Counting-Sort]
Using Figure 8.2 (or lecture note diagram) as a model, illustrate the operation of COUNTING-SORT on the array $A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$.

R2. (Exercise 8.3-1 on p. 199 of [CLRS]) [Exercising Radix-Sort]
[First, read up on Radix-Sort in text or from lecture notes]
Using Figure 8.3 (or lecture note diagram) as a model, illustrate the operation of RADIX-SORT on the following list of English words: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX.

R3. [Finding Min-and-Max simultaneosly]
This problem deals with finding both the *minimum and maximum* of n elements using as few comparisons as possible. One simple way is to just call Find-Min and Find-Max separately, which will take $2(n-1)$ comparisons. But, we can do better.

Given an algorithm to find both the *minimum and maximum* of n elements using as most $3\lceil n/2 \rceil$ comparisons (in the worst case).

[*Hint:* Try to solve this on your own first. If you get stuck, see [CLRS]-C9.1.]

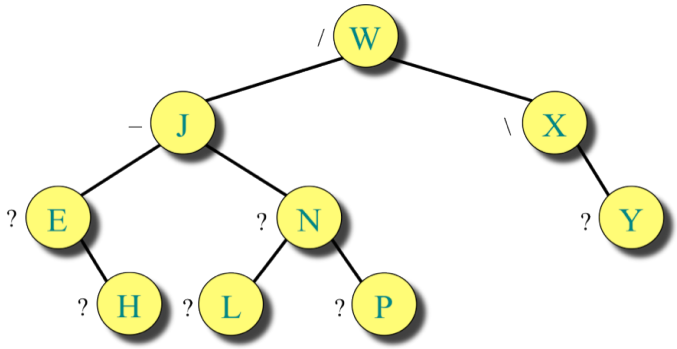
D-Problems: Solve these D-problems and prepare to discuss them in tutorial class. You may be called upon to present your solution *or your best attempt at a solution*. Your solution presentation does NOT need to be fully correct, given your best attempt. The TA will help clarify and correct any issues or errors.

D1. [Tri-Partitioning for a better Quicksort?]
In *standard* Quicksort, we partition an array $A[1..n]$ about a pivot element x. Prof. Triple suggests that we might be able to do better if we use *two* pivot elements, instead of just one. (He thinks that a good *tripartite partition* works better, just like the *good tripartite industrial relations* in Singapore.)

(a) Suppose that you choose *two* elements x and y (for simplicity, assume that $x < y$) to partition the array $A[1..n]$ into three partitions: the left partition with all elements $\leq x$, the middle partition with all elements $\geq x$ and $\leq y$, and the right partition with all elements $\geq y$. Give an algorithm to perform this tri-partitioning with two pivots.
[*Hint:* Extend/Generalize the partitioning algorithm given in the lectures (L04b-S4) to do partitioning into three parts, small-elements, medium-elements, large-elements.]

(b) What is the best-case for tri-partitioning. In that case, give the corresponding recurrence and solve it to get the best-case running time.

D2. [Augmenting DS: An OS Tree made from an AVL Tree]
Consider the following binary search tree *T*, with the *keys* shown inside the node. The tree *T* is also an AVL-tree with the "*balance*" information { /, –, \ } indicated next to the node. The *balance* information for the top three nodes are filled in already.



(a) Your first task is to fill in the balance information for all the other nodes in the tree. [If you have not learnt AVL trees before, read the uploaded notes on "AVL trees".]

(b) Next, show how to augment the AVL-tree *T* into an OS Tree with the size information (as defined in Lectures on "Augmenting Data Structures". Show the size information below each node in the tree.

(c) Show how the call OS-SELECT (*T*.root, 4) operates on the augmented AVL-tree *T*.

(d) Show how the call OS-RANK (*T*.root, Y) operates on the augmented AVL-tree *T*.

(e) Show how the call OS-INSERT (*T*.root, I) operates on the augmented AVL-tree *T*.

(f) Show how the call OS-DELETE (*T*.root, Y) operates on the *original* augmented AVL-tree *T*.

D3. Exercise 9.1-1 (p.215)
Second largest of n elements in $(n + \lceil \lg n \rceil - 2)$ comparisons in the worst case.

D4. [Alternative Decision Tree]
In the decision tree (for sorting 3 elements) shown in the lectures (Slide L05-S5.4]), we compare 1:2 (namely, a_1:a_2) at the root of the tree.

Give an alternative decision tree that compares 2:3 (namely, a_2:a_3) at the root node of the tree and also have *height* 3 (meaning that the worst-case time to sort is still 3 comparisons).